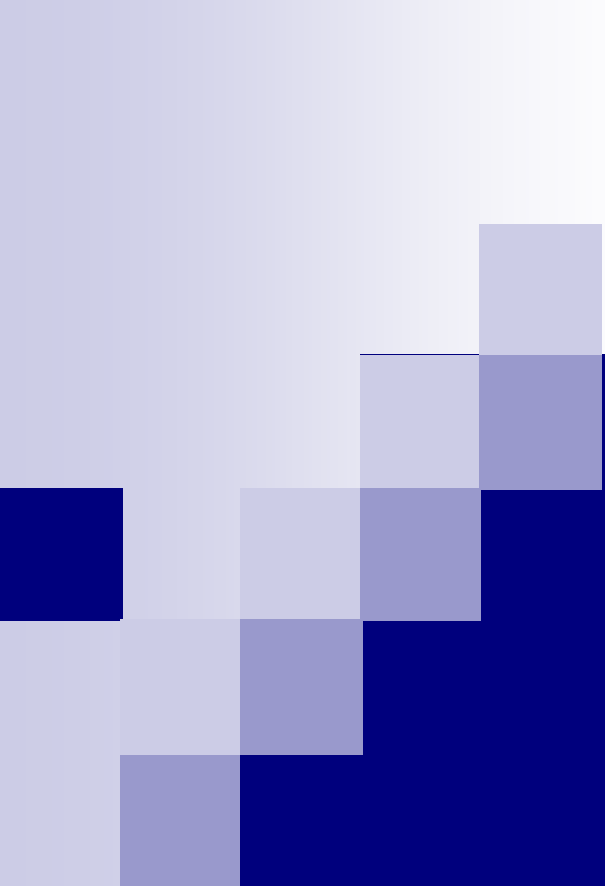




Modern Operating Systems

Chapter 1 – Introduction-2

(1.5-1.7)

Zhang Yang

cszyang@scut.edu.cn

Spring 2025



Contents of this lecture

- 1.5 Operating System Concepts
- 1.6 System Calls
- 1.7 Operating System Structure
- Summary

- 
- 1.5 Operating System Concepts
 - 1.6 System Calls
 - 1.7 Operating System Structure
 - Summary



OS Concepts

- Every modern operating systems provides support for:
 - Process Management
 - Memory Management
 - Management of Persistent Storage (File)
 - Communicating with peripherals (Input/Output)
 - Protection
 - User Interface (Shell)

Processes (1)

■ Process

- A process is a program in execution.
- A process is a running instance of a program.

■ Associated with each process are:

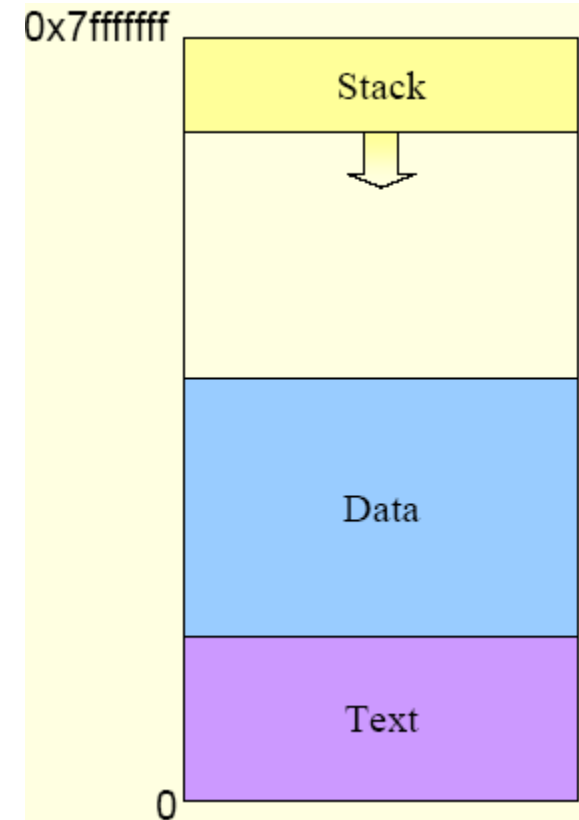
- The process's address space (instructions, data, stack)
- The values stored in a set of registers including program counter, stack pointer, and other hardware registers.
- Other information needed to run the program.

■ Each person authorized to use a system is assigned a UID (User Identification). Each process is uniquely identified with a process id.

Processes (2)

■ Memory Image of a Unix Process

- Text: program code
- Data: program data
 - Statically declared variables
 - Areas allocated by malloc() or new (heap)
- Stack
 - Automatic variables
 - Procedure call information
- Address space growth
 - Text: doesn't grow
 - Data: grows “up”
 - Stack: grows “down”



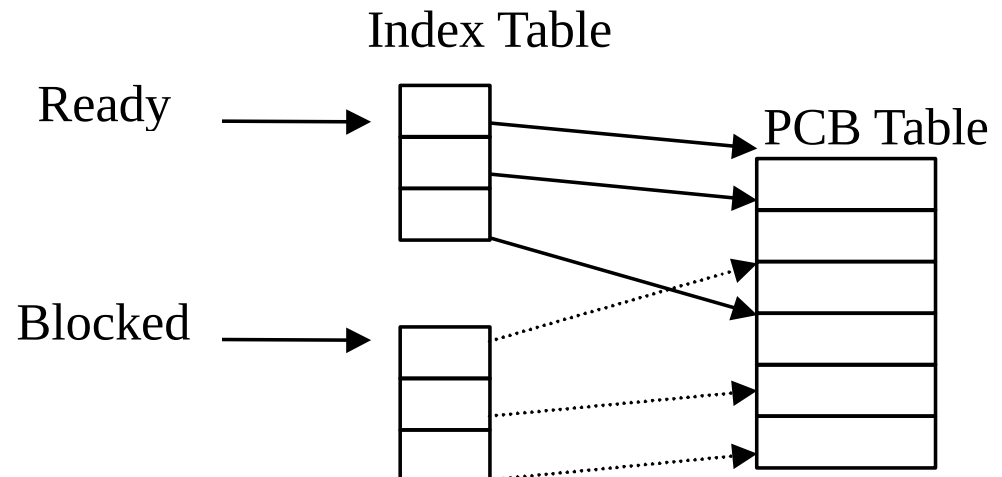
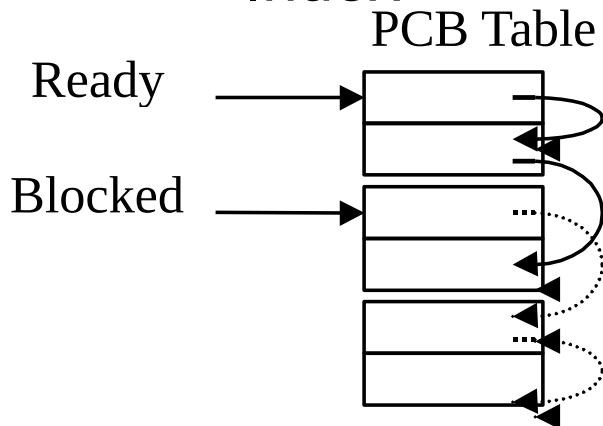
Processes (3)

■ Process Table / PCB Table

- Process Control Block is a data structure in the operating system kernel containing the information needed to manage a particular process.
- OS maintains a PCB for each process.
- OS keeps track of all processes in a *process table*.
- Two Organization Form

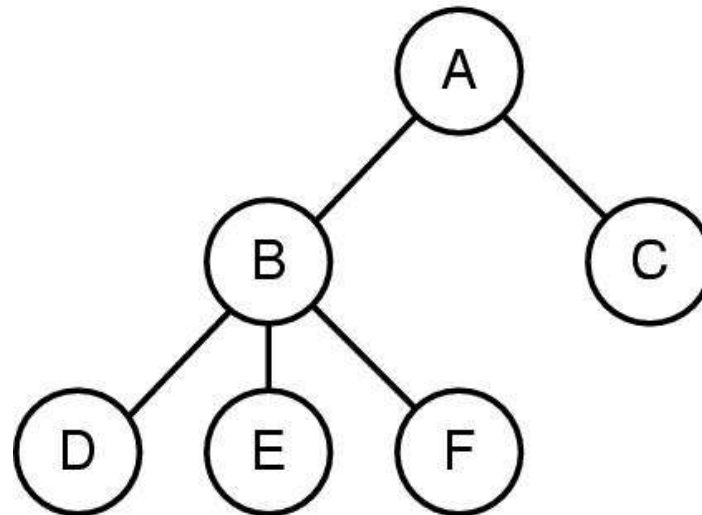
■ Link

■ Index



Processes (4)

- Processes can create other processes
 - A process tree
 - A created two child processes, B and C
 - B created three child processes, D, E, and F





Processes (5)

- Process Management Activities
 - Creating and deleting both user and system processes.
 - Suspending and resuming processes.
 - Providing mechanisms for process synchronization.
 - Providing mechanisms for process communication.
 - Providing mechanisms for deadlock handling.

Address Space (1)

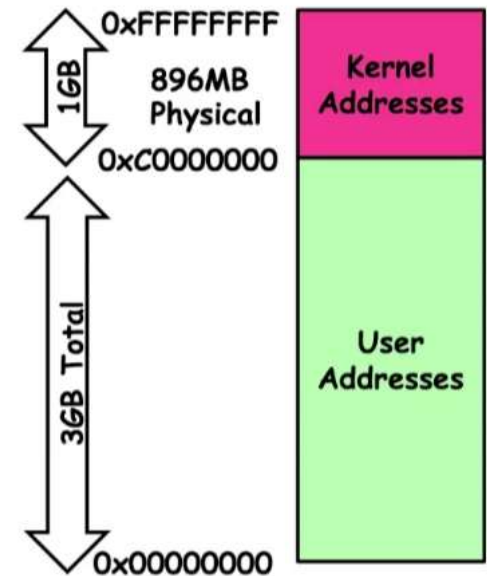
- Memory management is the functionality of an operating system which handles or manages **primary memory** and moves processes back and forth between main memory and disk during execution.
- Managing and Protecting of Main Memory
 - Keeping track of which parts of memory are currently being used and by whom.
 - Deciding which processes (or parts thereof) and data to move into and out of memory.
 - Allocating and deallocating memory space as needed.

Address Space (2)

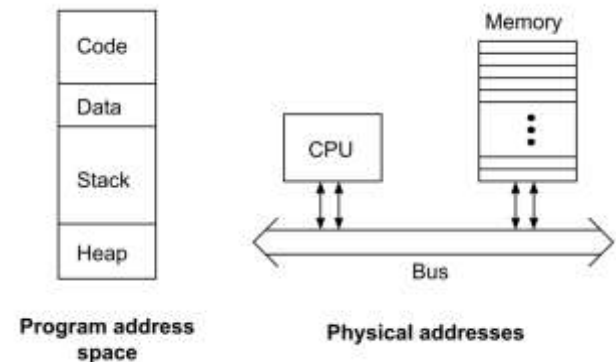
■ Process Address Space

- The address space is how the process sees its own memory.
- Every process has its own **separate** address space.
 - Memory addresses are “private” to each process.
 - If Process A accesses address 0x3000, and Process B accesses 0x3000, they are accessing **different** physical memory locations.
 - *Unless, of course, the processes have been set up to share memory somehow.*

□ Virtual Memory



32-Bit Virtual Address Space



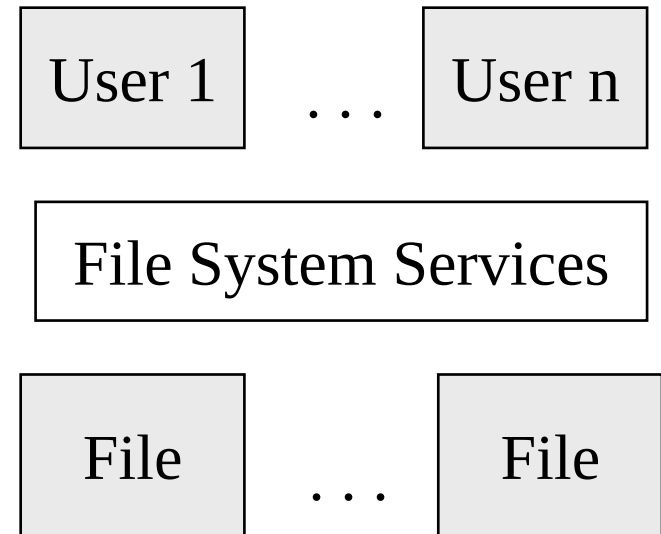


File Management (1)

- Files, Directories and File Systems
 - Secondary storage provides persistent storage in a computer system.
 - OS implements an abstraction that is called a file to represent bytes stored on secondary storage.
 - Files are organized into directories.
 - Directories make up file systems.

File System (3)

- The OS provides services for managing files, directories, and file systems on secondary storage.
- OS activities include
 - Creating and deleting files and directories.
 - Primitives to manipulate files and directories.
 - Mapping files onto secondary storage.
 - Backup files onto stable (non-volatile) storage media.



File System (3)

- File system for a university department

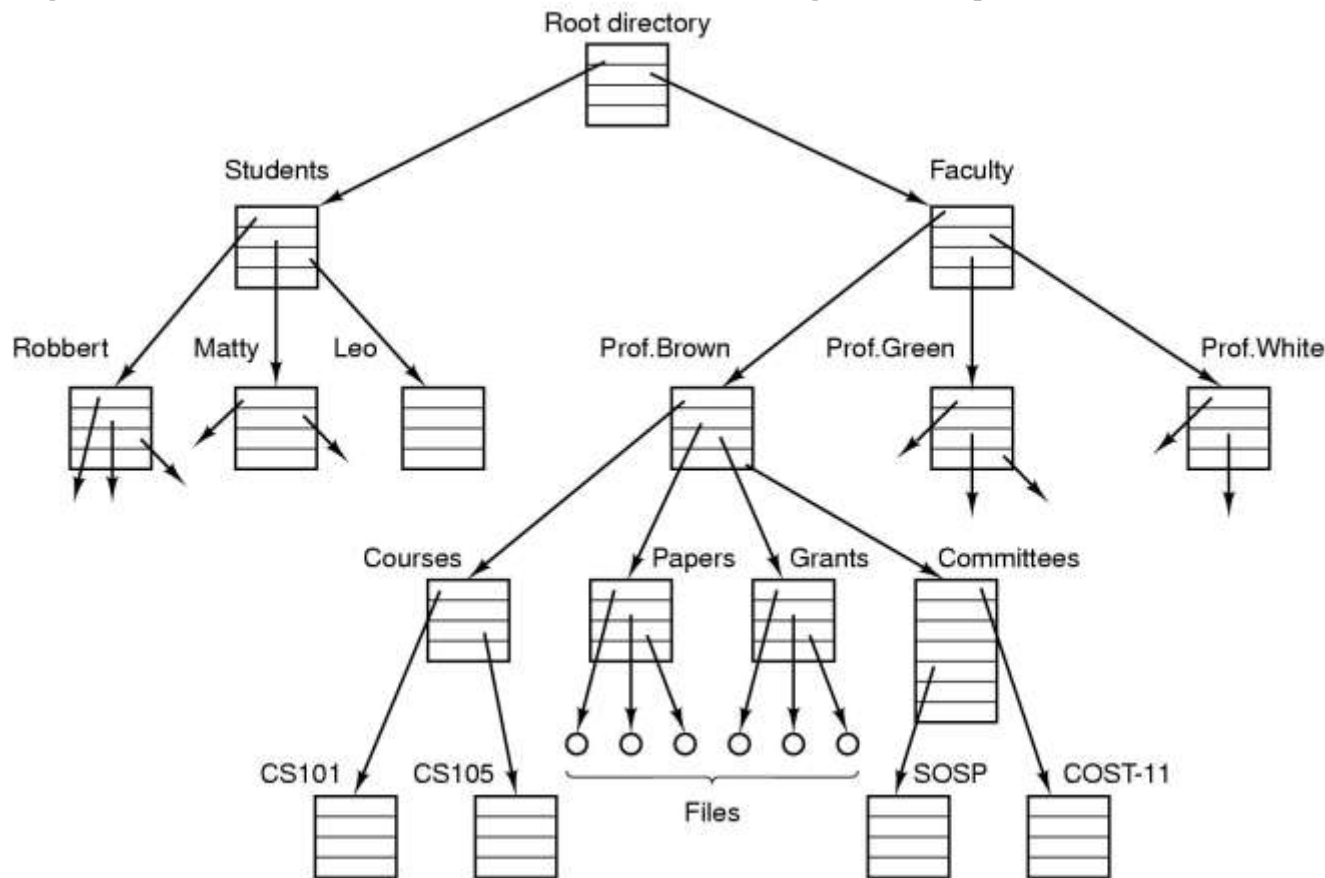
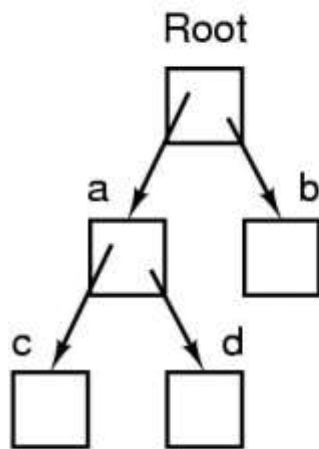


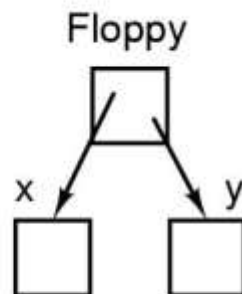
Figure 1-14. A file system for a university department.

File System (4)

- Unix file systems can be extended to include files from additional secondary storage devices.
 - Before mounting
 - Files on floppy are inaccessible.
 - After mounting floppy on b,
 - Files on floppy are part of file hierarchy.



(a)



(b)

File System (5)

■ Unix Special Files

□ Special Files (Device Files)

- Represent hardware or logical devices.
- An interface for a device driver that appears in a file system as if it was an ordinary file.
- Found in directory called /dev.

□ Block Special File (Block Device)

- Talks to devices 1 block at a time (1 block = 512 bytes to 32KB)
- Hard disk, DVD, CD ROM, ...
- e.g. /dev/hd1

File System (6)

■ Unix Special Files (ctd.)

□ Character Special File (Character Device)

- Used to model serial I/O devices such as keyboards and printers.
- Talks to devices in a character by character (1 byte at a time)
- e.g. /dev/lp

File System (7)

■ Pipe

- A pipe provides a communication channel between two independent processes.

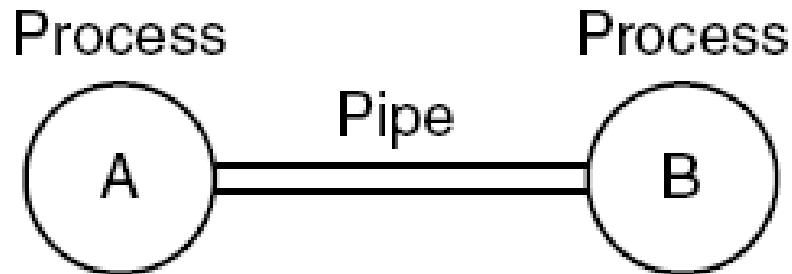


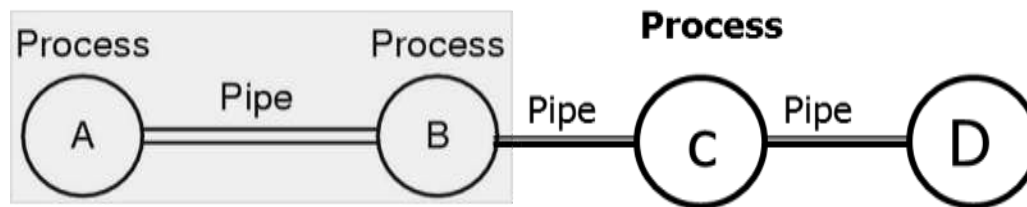
Figure 1-16. Two processes connected by a pipe.

- One process writes to the pipe, the other reads from the pipe.
- Looks exactly the same as reading from/to a file.

File System (8)

■ Pipe (ctd.)

- Sometimes useful to connect a set of processes in a pipeline.



- Process A writes to pipe AB, Process B reads from AB and writes to BC
- Process C reads from BC and writes to CD

Input/Output (1)

- Every OS has an I/O sub-system.
- OS must understand how to communicate with many types of peripherals (keyboard, mouse, monitor, printer,...)
- The OS must provide an interface to I/O devices that enables programmers to perform I/O without being concerned with H/W details.
- Modern operating systems interface with the hardware through **device drivers**.

Input/Output (2)

■ Device Driver

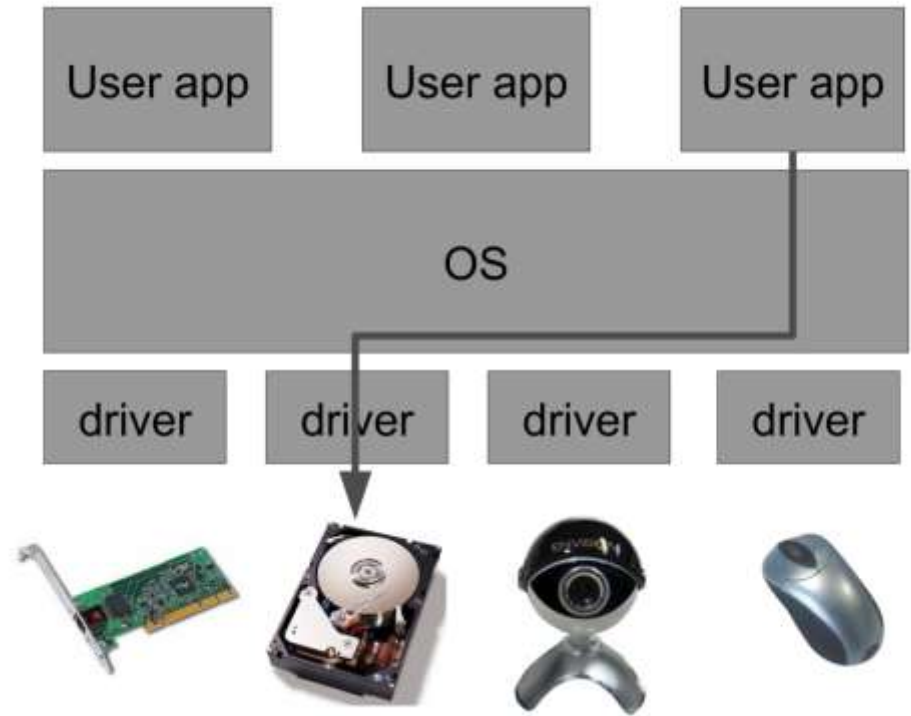
- Black boxes to hide details of hardware devices.

- Main Role

- Map standard calls to device-specific operations.

- Can be developed separately from the rest of the kernel.

- Plugged in at runtime when needed.





Protection (1)

- OS must prevent unauthorized access too resources.
- Must also prevent one user from interfering with another user's resources (e.g. files).
- OS provides mechanisms for
 - Determining who can gain access to the computer system.
 - What resources a user can access.
 - What action(s) the user can carry out on resource.

Protection (2)

■ Protection Example

- Files in UNIX are protected by a 9-bit binary protection code.
 - The protection code consists of three 3-bit fields, one for owner, one for group, and one for others.
 - Each field has a bit for read, a bit for write, and a bit for execute. They are known as rwx bits.
 - e.g. `rwXr-x--X`



Kernel & Utility

■ Kernel

- Usually refers to that part of the OS that implements basic functionality and is always present in memory.
- In some cases the entire OS is created as one monolithic entity and this entire unit is called the kernel.

■ Utility

- Programs that are not part of the OS kernel, but work closely with the kernel to provide ease of use or access to system information.
- E.g., Shell, File Management Tool, Disk Management Tool

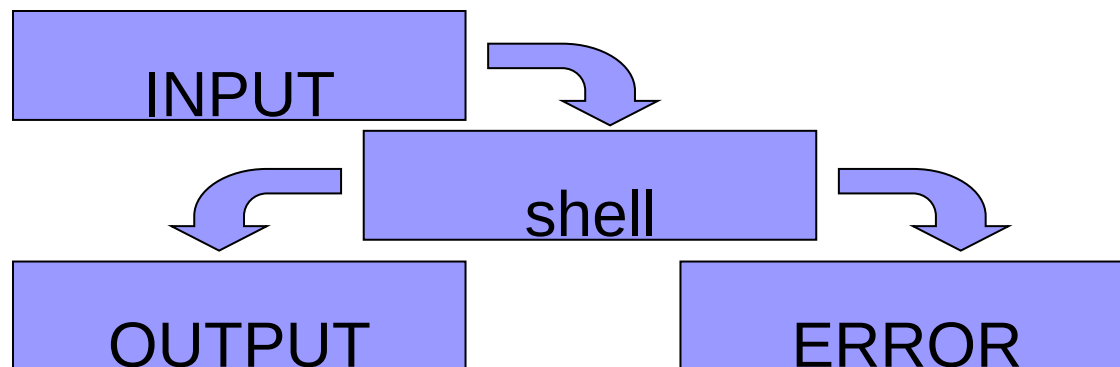
Shell (1)

■ What is Shell?

- A shell is software that provides an interface for an operating system's users to provide access to the kernel's services.

■ Uses of a Shell

- One of the major uses of a shell is to interpret commands entered at the prompt.



Shell (2)

■ Uses of a Shell (ctd.)

- Another important function of the shell is to customize the user's environment, normally done in shell initialization files.
 - Definitions for setting terminal keys and window characteristics.
 - Setting variables that define the search path, permissions, prompts, and the terminal type.
 - Setting variables that are required for specific applications such as windows, text-processing programs, and libraries for programming languages.

Shell (3)

■ Uses of a Shell (ctd.)

- The shell can also be used as an interpreted programming language.
 - Shell programs, also called scripts, consist of commands listed in a file.
 - Consist of Linux commands interspersed with fundamental programming constructs, such as variable assignment, conditional tests, and loops.

Shell (4)

- Different Types of Shell in Linux
 - Bourne Shell: sh
 - The first UNIX shell
 - GNU Bourne Shell: bash
 - C Shell: csh
 - Shell with C-like syntax
 - Better for user customization and scripting
 - Korn Shell: ksh
 - Superset of sh
 - Z Shell
 - zsh

Shell (5)

■ Different Types of Shell in Linux (ctd.)

Shell	Complete path-name	Prompt for root user	Prompt for non root user
Bourne shell (sh)	/bin/sh and /sbin/sh	#	\$
GNU Bourne-Again shell (bash)	/bin/bash	bash-VersionNumber#	bash-VersionNumber\$
C shell (csh)	/bin/csh	#	%
Korn shell (ksh)	/bin/ksh	#	\$
Z Shell (zsh)	/bin/zsh	<hostname>#	<hostname>%

Shell (6)

■ An Example of Bourne Shell

```

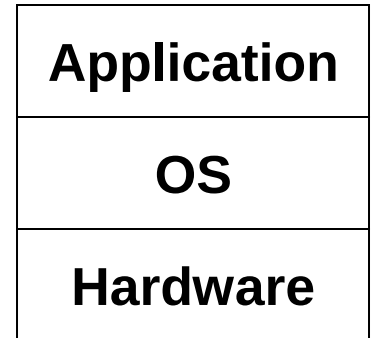
PBG-Mac-Pro:~ pbg$ w
15:24 up 56 mins, 2 users, load averages: 1.51 1.53 1.65
USER      TTY      FROM            LOGIN@   IDLE   WHAT
pbg       console  -               14:34    50    -
pbg       s000    -               15:05    -    w
PBG-Mac-Pro:~ pbg$ iostat 5
            disk0            disk1            disk10            cpu            load average
      KB/t tps MB/s      KB/t tps MB/s      KB/t tps MB/s  us sy id  1m  5m  15m
      33.75 343 11.30      64.31 14  0.88      39.67  0  0.02  11  5 84  1.51 1.53 1.65
       5.27 320  1.65        0.00  0  0.00        0.00  0  0.00   4  2 94  1.39 1.51 1.65
       4.28 329  1.37        0.00  0  0.00        0.00  0  0.00   5  3 92  1.44 1.51 1.65
^C
PBG-Mac-Pro:~ pbg$ ls
Applications          Music                  WebEx
Applications (Parallels)  Pando Packages       config.log
Desktop               Pictures              getsmartdata.txt
Documents             Public                imp
Downloads             Sites                 log
Dropbox              Thumbs.db            panda-dist
Library              Virtual Machines     prob.txt
Movies              Volumes              scripts
PBG-Mac-Pro:~ pbg$ pwd
/Users/pbg
PBG-Mac-Pro:~ pbg$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1): 56 data bytes
64 bytes from 192.168.1.1: icmp_seq=0 ttl=64 time=2.257 ms
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=1.262 ms
^C
--- 192.168.1.1 ping statistics ---
2 packets transmitted, 2 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 1.262/1.760/2.257/0.498 ms
PBG-Mac-Pro:~ pbg$
```

- 
- 1.5 Operating System Concepts
 - 1.6 System Calls
 - 1.7 Operating System Structure
 - Summary

From Slide What is OS

■ Code that:

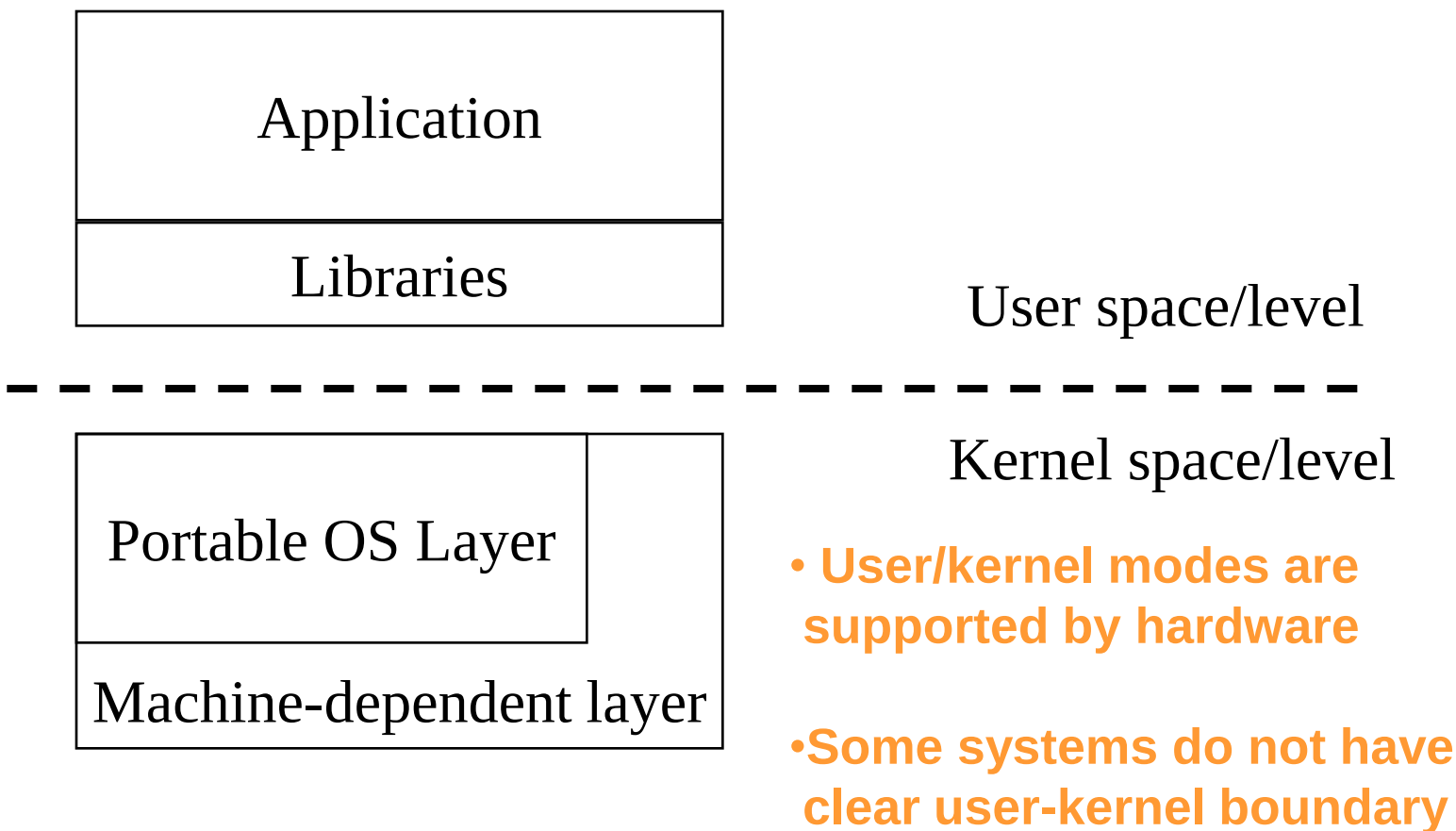
- Sits between programs & hardware
- Sits between different programs
- Sits between different users



■ Job of OS:

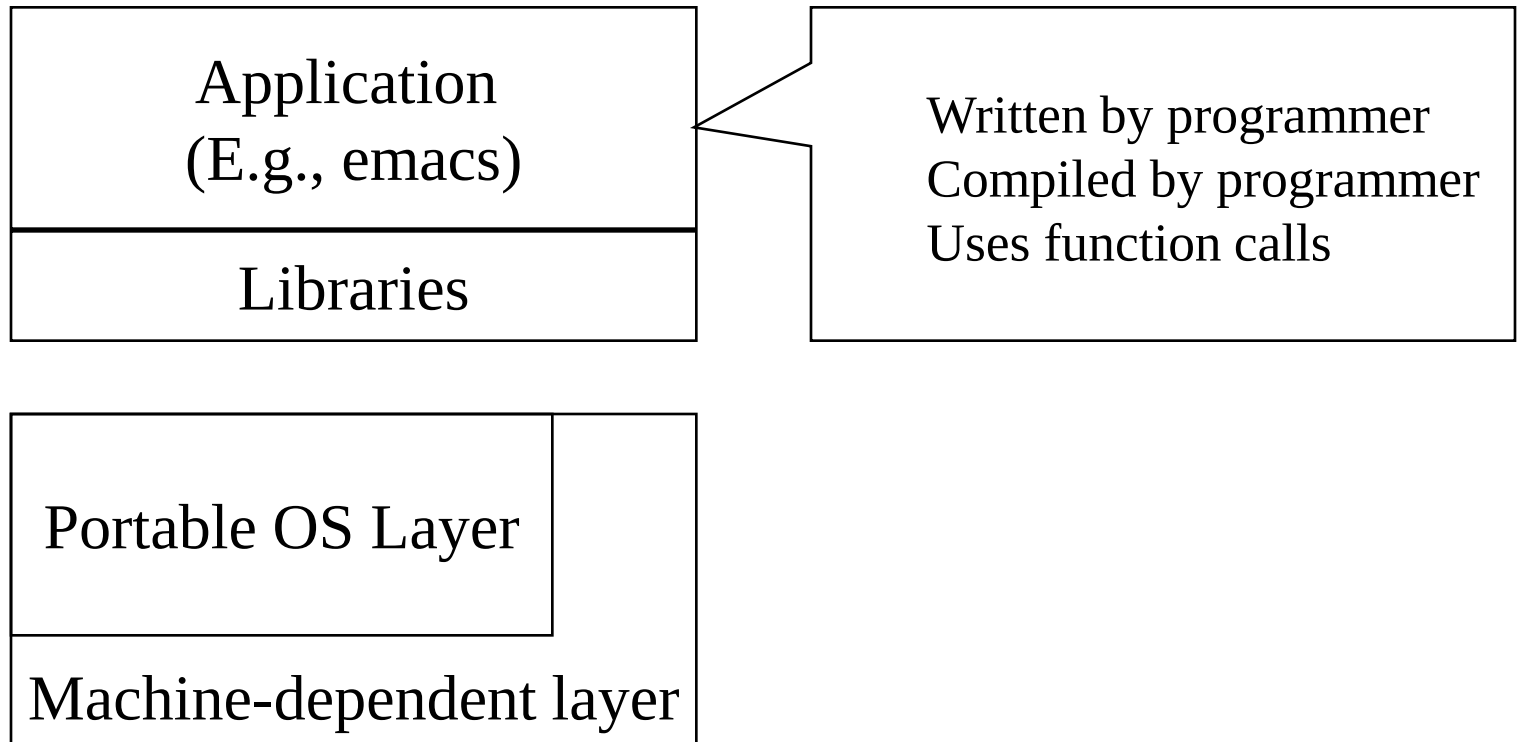
- Manage hardware resources
 - Allocation, protection, reclamation, virtualization
- Provide services to app. How? -- System Call
 - Services are functions that the OS kernel provides to users.
 - Abstraction, simplification, standardization

A Peek into Unix/Linux (1)



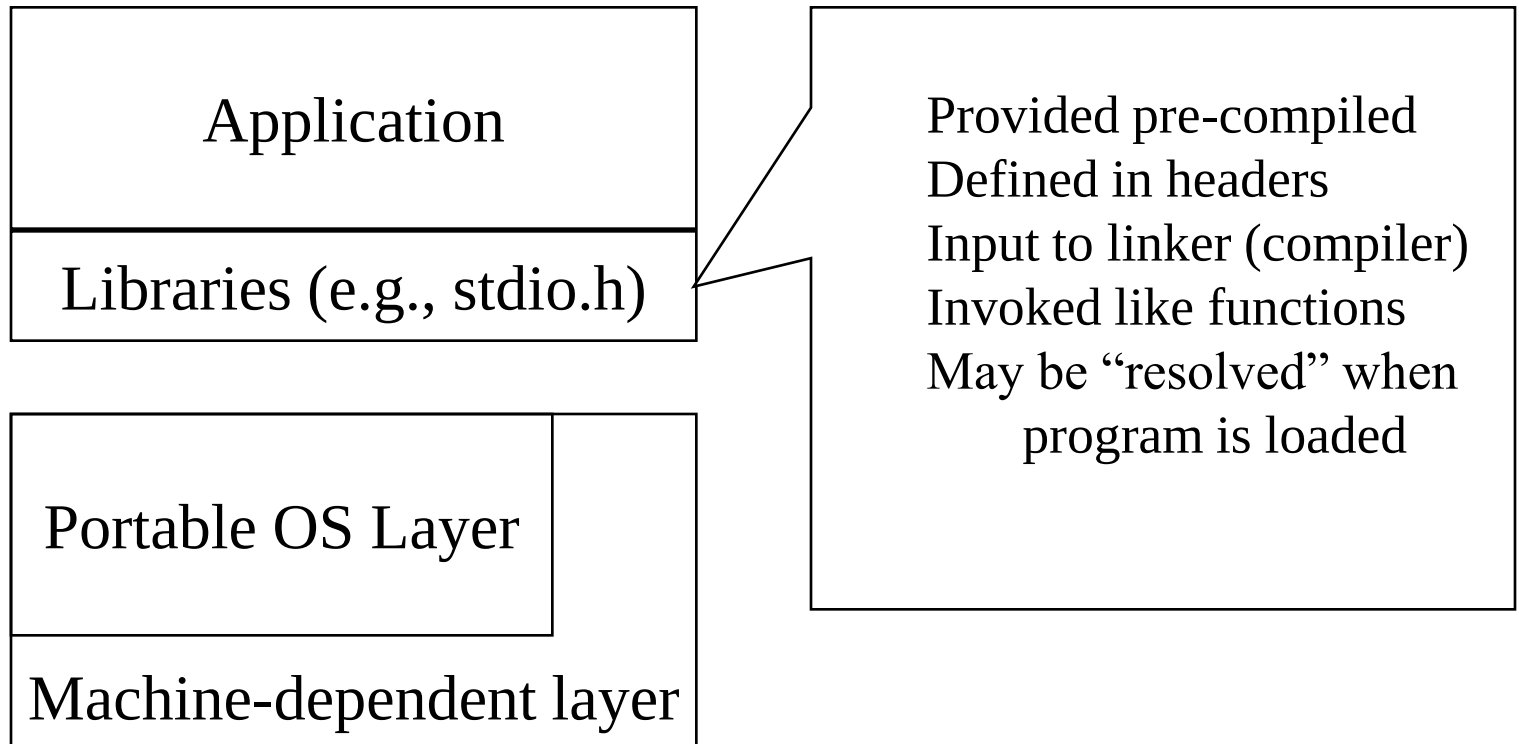
A Peek into Unix/Linux (2)

■ Unix: Application



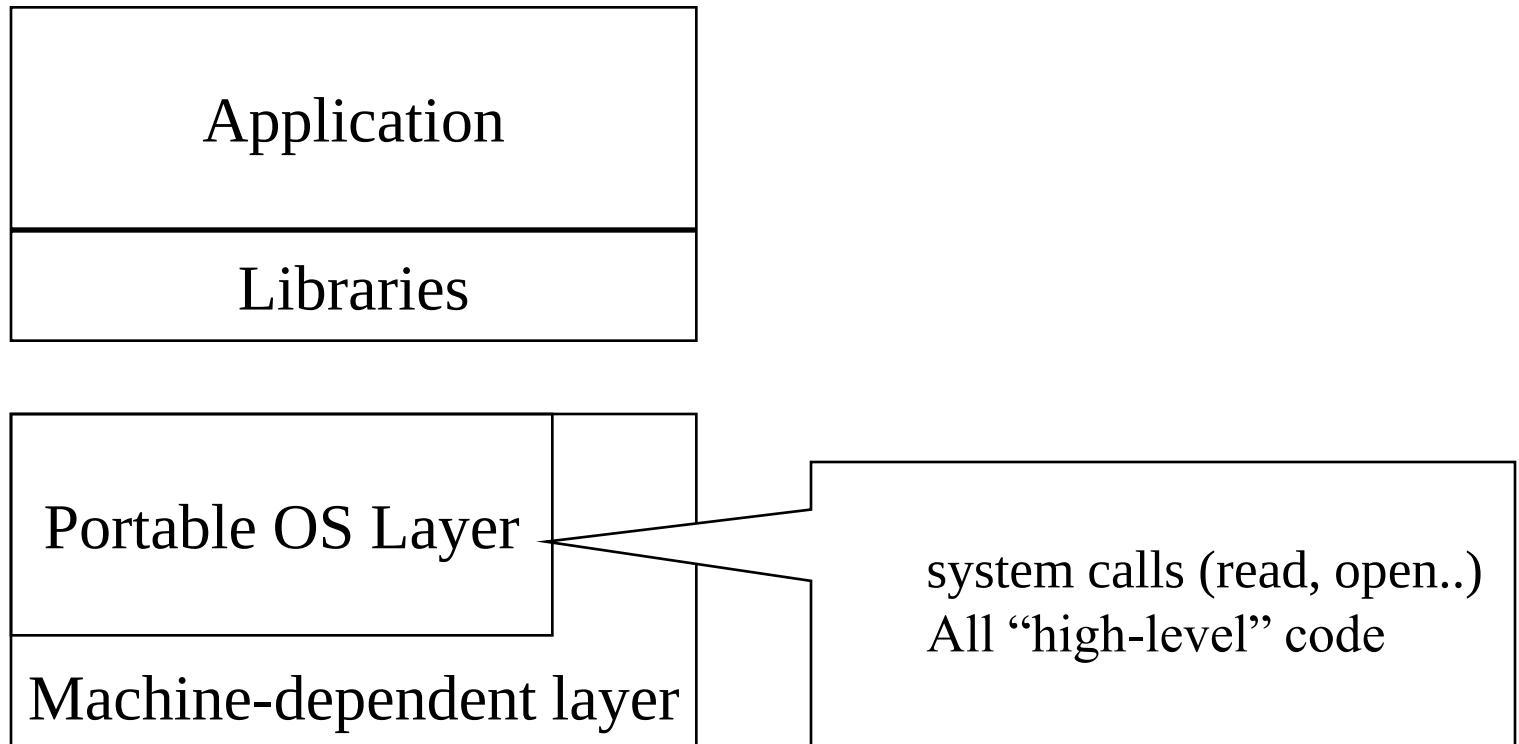
A Peek into Unix/Linux (3)

■ Unix: Libraries



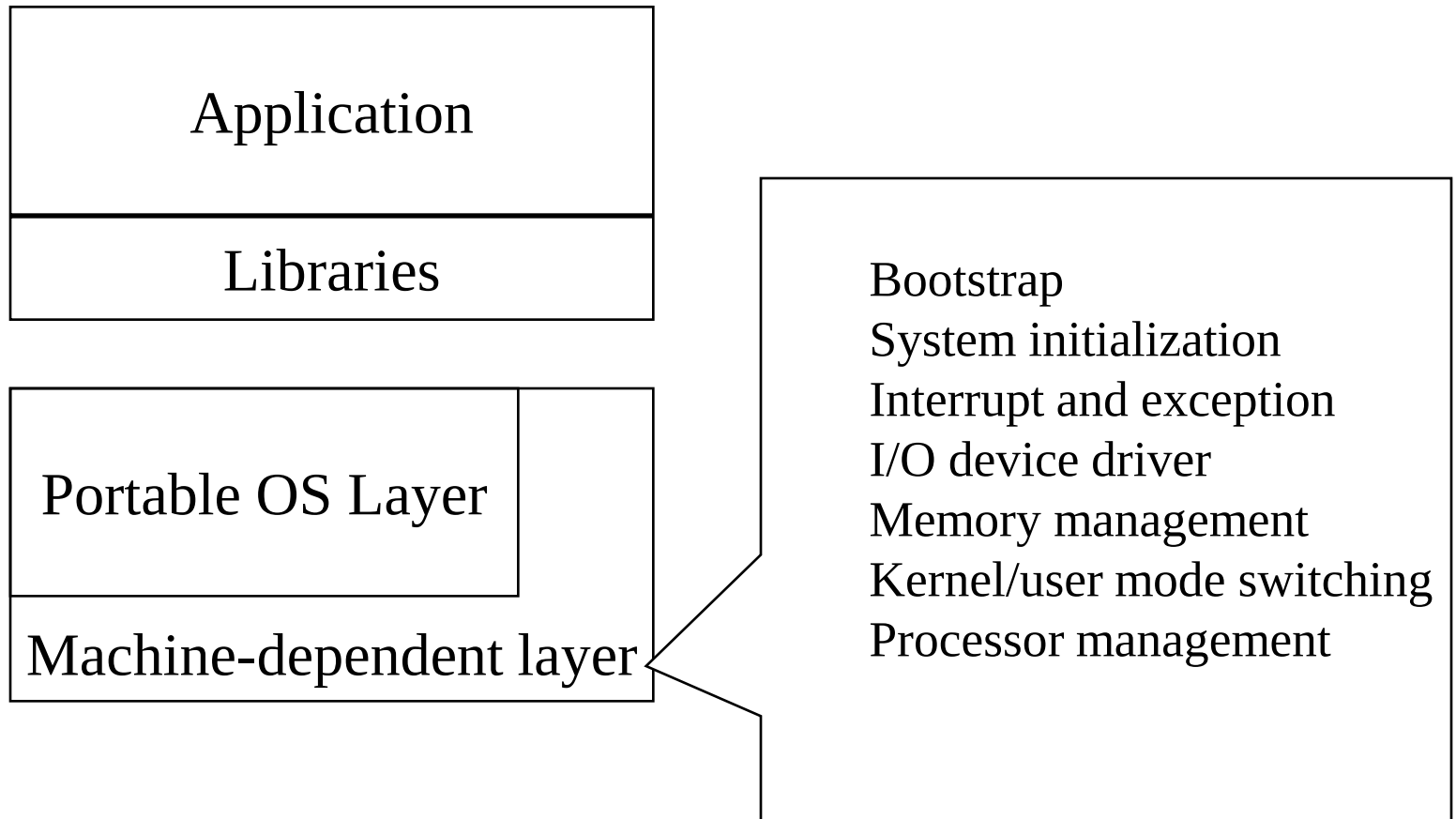
A Peek into Unix/Linux (4)

■ Typical Unix OS Structure



A Peek into Unix/Linux (5)

■ Typical Unix OS Structure (ctd.)



System Calls (1)

■ System Calls

- Programming interface to the services provided by the OS.
- Typically written in a high-level language (C or C++).
- Mostly accessed by programs via a high-level Application Programming Interface (API) rather than direct system call use.
- System calls often use a special machine code instruction which causes the processor to change mode (e.g. to "kernel mode" or "protected mode").
- Three most common APIs
 - **Win32 API** for Windows
 - **POSIX API** for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X)
 - **Java API** for the Java virtual machine (JVM)

System Calls (2)

■ Example: read (fd, buffer, nbytes)

- fd: file descriptor
- nbytes: number of bytes to read

count=read (fd,
buffer,nbytes)

System Call Code

linux4.20.4

```
\arch\sh\include\uapi\asm\unistd_64.h
```

```
#define  
__NR_restart_syscall    0  
#define __NR_exit        1  
#define __NR_fork        2  
#define __NR_read        3  
#define __NR_write       4
```

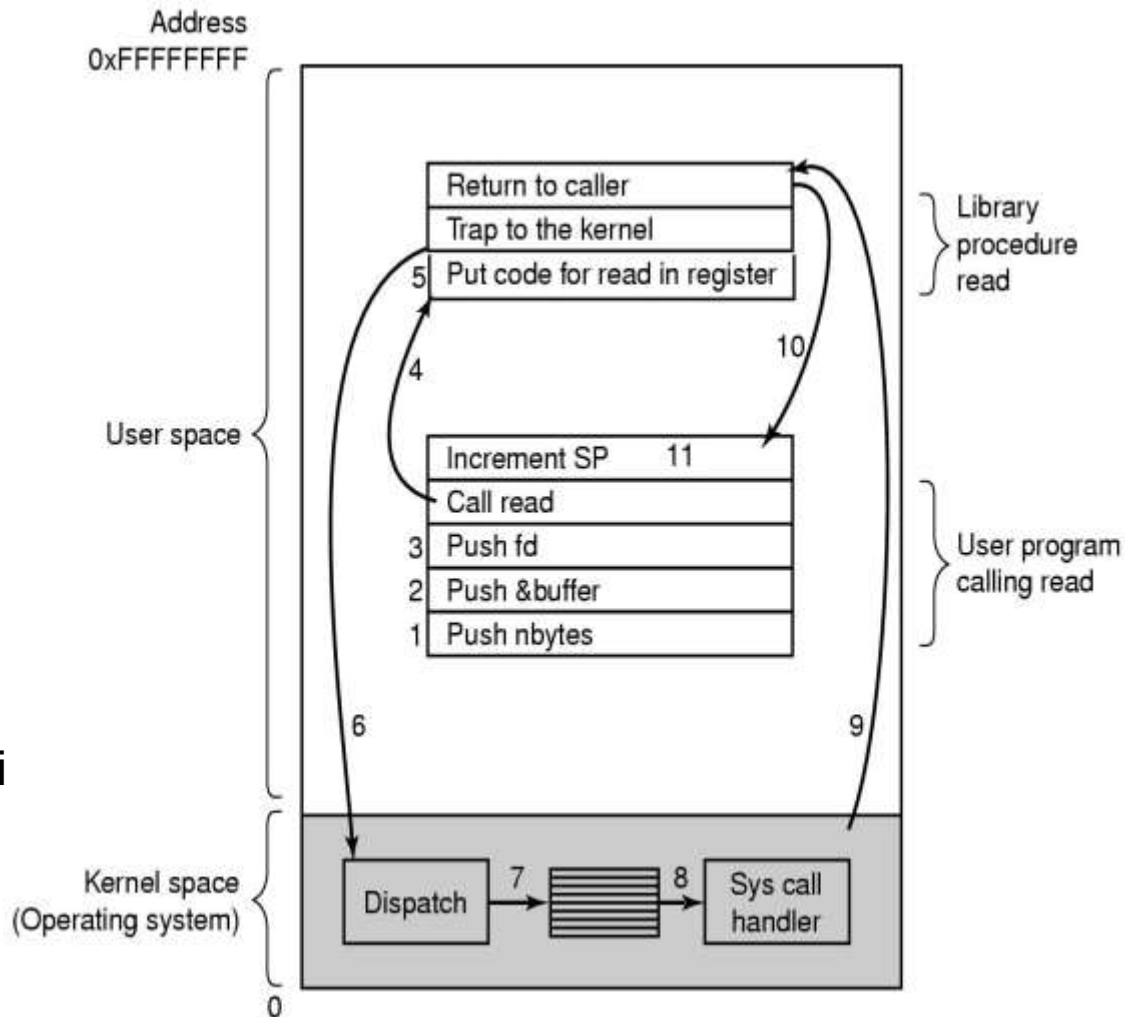


Figure 1-17. Steps in Making a System Call

System Calls (3)

■ POSIX

- Portable Operating System Interface for uniX
- Set of IEEE standards
- Mandatory + Optional Parts
- Objective: Source code portability of applications across multiple OS.
 - Standard way for applications to interface to OS.
 - Mostly but not exclusively Unix type OS.
 - *Total portability is not achievable.*

- The mapping of POSIX procedure calls onto system calls is not one-to-one.

System Calls (4)

■ System Calls for Process Management

Process management

Call	Description
<code>pid = fork()</code>	Create a child process identical to the parent
<code>pid = waitpid(pid, &statloc, options)</code>	Wait for a child to terminate
<code>s = execve(name, argv, environp)</code>	Replace a process' core image
<code>exit(status)</code>	Terminate process execution and return status

□ fork ()

- fork is the only way to create a new process in POSIX.
- The new process created by *fork* is called the *child process*.
- This function is called once but returns twice.
 - Returns: 0 in child, process ID of child in parent, -1 on error
- Note: Both the child and parent continue executing with the instruction that follows the call to *fork*.

System Calls (5)

■ System Calls for Process Management (ctd.)

□ `waitpid()`

- Return: process ID if OK, 0 or -1 on error
- The interpretation of the *pid* argument depends on its value:
 - *pid* == -1 waits for any child process. In this respect, *waitpid* is equivalent to *wait*.
 - *pid* > 0 waits for the child whose process ID equals *pid*.

System Calls (6)

■ System Calls for Process Management (ctd.)

□ execve()

■ Function Prototype

□ int execve (const char *filename,
char *const argv[], char *const envp[]);

- execve function allows child process to execute code that is **different** from that of parent.
- exec family of functions (*execl*, *execv*, *execle*, *execve*) provides a facility for overlaying the process image of the calling process with a new image.

System Calls (7)

■ System Calls for Process Management (ctd.)

□ Example of fork used in simplified shell program

```
#define TRUE 1
while (TRUE) {                                /* repeat forever */
    type_prompt( );                            /* display prompt */
    read_command (command, parameters)        /* input from terminal */
    if (fork() != 0) {                         /* fork off child process*/
        /* Parent code */
        waitpid( -1, &status, 0);             /* wait for child to exit */
    }
    else {
        /* Child code */
        execve (command, parameters, 0);      /* execute command */
    }
}
```

- Consider the case of a command such as

cp file1 file2

- The main program of cp: **main(argc, argv, envp)**

System Calls (8)

■ Some System Calls for File Management

File management

Call	Description
<code>fd = open(file, how, ...)</code>	Open a file for reading, writing or both
<code>s = close(fd)</code>	Close an open file
<code>n = read(fd, buffer, nbytes)</code>	Read data from a file into a buffer
<code>n = write(fd, buffer, nbytes)</code>	Write data from a buffer into a file
<code>position = lseek(fd, offset, whence)</code>	Move the file pointer
<code>s = stat(name, &buf)</code>	Get a file's status information

System Calls (9)

■ Some System Calls for Directory and File System Management

Directory and file system management

Call	Description
<code>s = mkdir(name, mode)</code>	Create a new directory
<code>s = rmdir(name)</code>	Remove an empty directory
<code>s = link(name1, name2)</code>	Create a new entry, name2, pointing to name1
<code>s = unlink(name)</code>	Remove a directory entry
<code>s = mount(special, name, flag)</code>	Mount a file system
<code>s = umount(special)</code>	Unmount a file system

System Calls (10)

- Some System Calls for Directory and File System Management (ctd.)
 - Example: `link("/usr/jim/memo", "/usr/ast/note")`

/usr/ast		/usr/jim	
16	mail	31	bin
81	games	70	memo
40	test	59	f.c.
		38	prog1

(a)

(a) Two directories before linking
/usr/jim/memo to ast's directory

/usr/ast		/usr/jim	
16	mail	31	bin
81	games	70	memo
40	test	59	f.c.
70	note	38	prog1

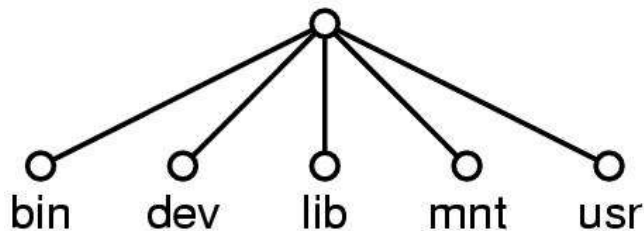
(b)

(b) The same directories after linking

System Calls (11)

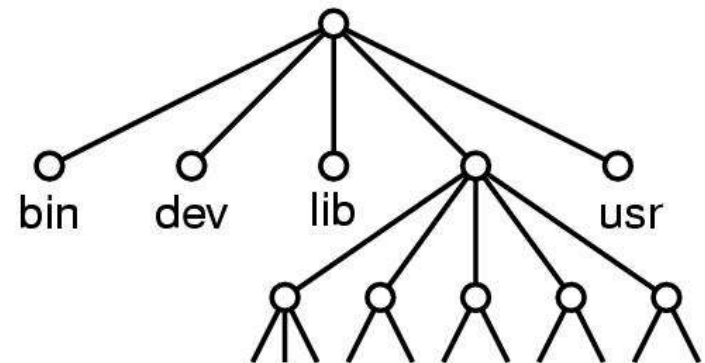
■ Some System Calls for Directory and File System Management (ctd.)

□ Example: `mount("/dev/sdb0", "/mnt", 0)`



(a)

(a) File system before the mount



(b)

(b) File system after the mount

System Calls (12)

■ Some System Calls for Miscellaneous

Miscellaneous

Call	Description
<code>s = chdir(dirname)</code>	Change the working directory
<code>s = chmod(name, mode)</code>	Change a file's protection bits
<code>s = kill(pid, signal)</code>	Send a signal to a process
<code>seconds = time(&seconds)</code>	Get the elapsed time since Jan. 1, 1970

- E.g. `chdir("/usr/ast/test");`
- E.g. `chmod("file",0644);` means to make file read only by everyone except the owner.



System Calls (13)

■ The Windows Win32 API

- In windows, the library calls and the actual system calls are highly decoupled.
- Win32 API for programmers to use to get OS services.
- The number of Win32 API calls is extremely large, numbering in the thousands.
- While many of Win32 APIs do invoke system calls, a substantial number are carried out entirely in user space.

System Calls (14)

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

Some Win32 API calls

- 
- 1.5 Operating System Concepts
 - 1.6 System Calls
 - 1.7 Operating System Structure
 - Summary

Operating System Structure (1)

■ Monolithic System - "the big mess"

- All operating system operations are put into a single file. The operating system is a collection of procedures, each of which can call any of the others.
- Basic Structure
 - A main program that invokes the requested service procedure.
 - A set of service procedures that carry out the system calls.
 - A set of utility procedures that help the service procedures.

Operating System Structure (2)

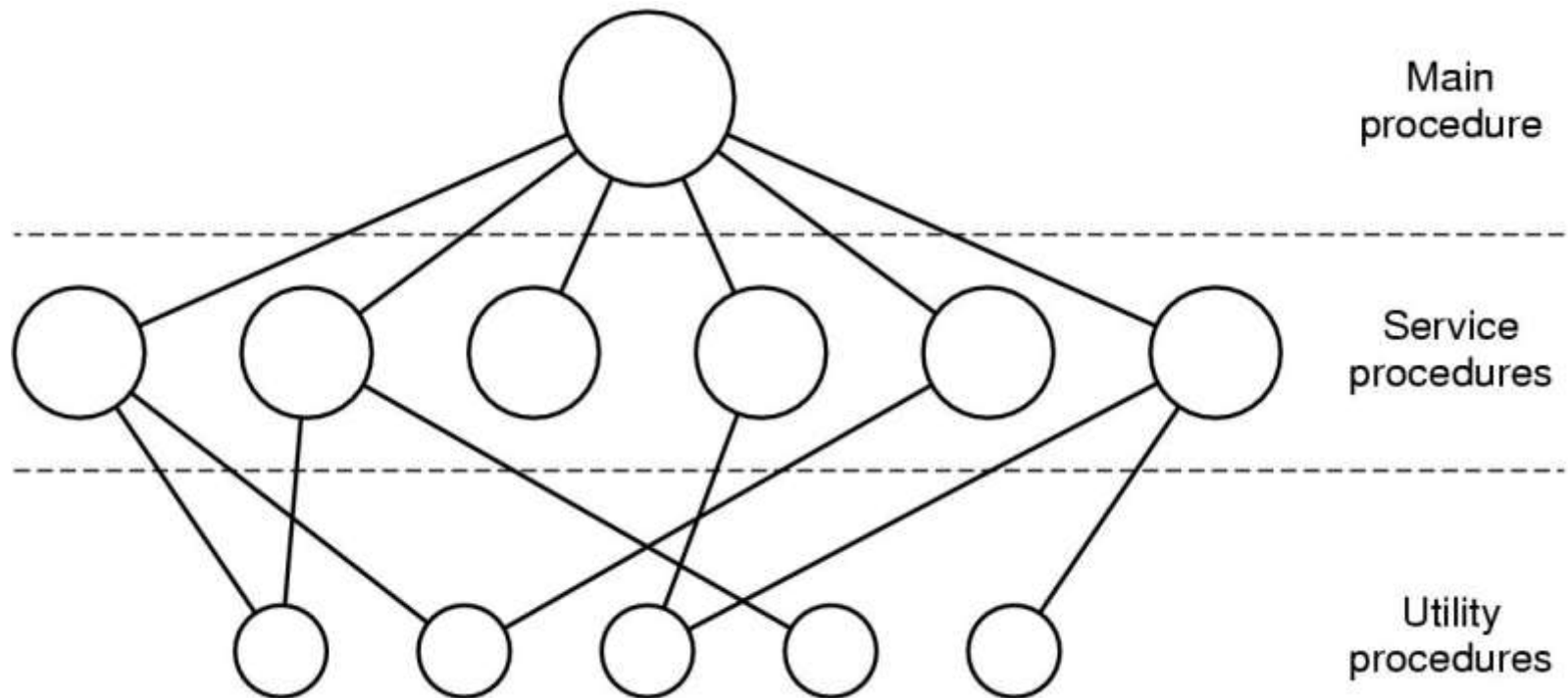


Figure 1-24. Simple structuring model for a monolithic system

Operating System Structure (3)

■ Monolithic System - "the big mess" (ctd.)

□ Examples

- BSD(Berkeley Software Distribution), FreeBSD, NetBSD, OpenBSD, MirOS BSD, SunOS.
- UNIX System V, AIX, HP-UX, Solaris, OpenSolaris / illumos.
- CP/M
- MS-DOS

□ Problems:

- Kernel components aren't protected from each other.
- Not easily extended/modified.
- Structure may be unclear.

Operating System Structure (4)

■ Layered System

- The operating system is divided into a number of layers (levels), each built on top of lower layers.
- The different services of the operating system are split into various layers, where each layer has a specific well-defined task to perform.
- The bottom layer (layer 0) is the hardware; the highest (layer N) is application with user interface.
- A particular layer can access all the layers present **below** it but it **cannot** access the layers present **above** it. That is layer $n-1$ can access all the layers from $n-2$ to 0 but it cannot access the n th layer.
- Hiding information at each layer.
- Develop a layer at a time.

Operating System Structure (5)

■ Layered System (ctd.)

□ THE Operating System (Dijkstra, 1968)

- A simple batch system for Electrologica X8
- All the parts of the system were ultimately linked together into a single exe.
- Main advantage: Each layer can be tested and verified independently

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

Figure 1-25. Structure of the THE operating system

Operating System Structure (6)

■ Layered System (ctd.)

□ Advantages

■ Modularity

- Promote modularity as each layer performs only the tasks it is scheduled to perform.

■ Easy debugging

- Debug the particular layer that has error.

■ Easy update

- A modification made in a particular layer will not affect the other layers.

■ No direct access to hardware

■ Abstraction



Operating System Structure (7)

■ Layered System (ctd.)

□ Disadvantages

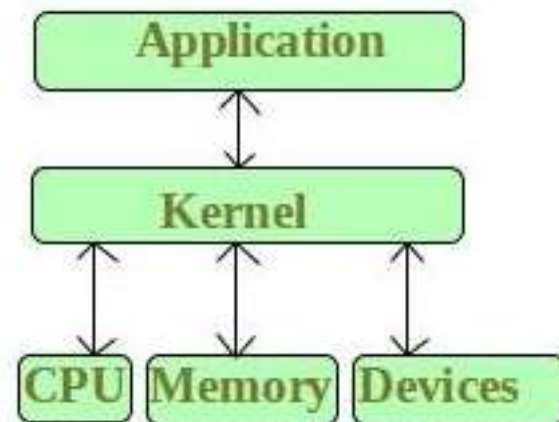
- Complex and careful implementation.
 - The arrangement of the layers must be done carefully.
 - With great modularity comes complex implementation.
- Slower in execution.

Operating System Structure (8)

■ Microkernels

□ Kernel

- The core part of an operating system which manages system resources.
- It also acts like a bridge between application and hardware of the computer.
- It is one of the first programs loaded on start-up (after the Bootloader).



Operating System Structure (9)

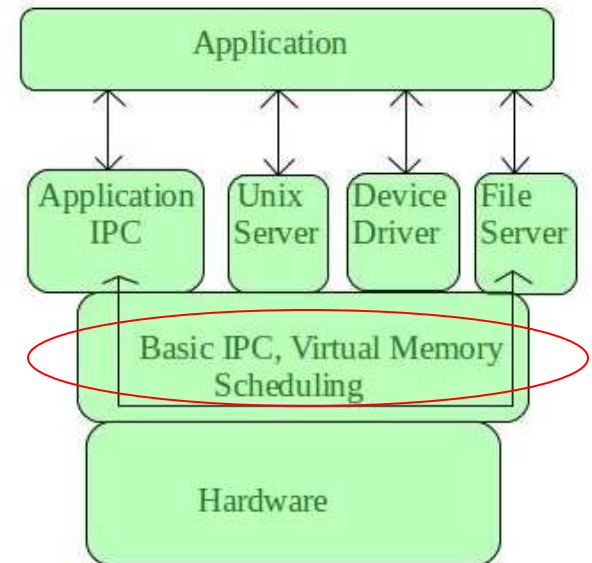
■ Microkernels (ctd.)

□ Microkernel

- Move as much functionality as possible from the kernel into “user” space.
- OS kernel is very small – minimal functionality.
 - Primitive memory management (address space)
 - I/O and interrupt management
 - Inter-Process Communication (IPC)
 - Basic scheduling
- Other OS functions provided at user level by **trusted** servers.
 - User process, trusted by kernel.
 - Device drivers, file system, virtual memory.

□ Better Known Microkernels:

- Mach, K42, L4, MINIX 3



Operating System Structure (10)

- Microkernels (ctd.)

- MINIX 3

- Reincarnation Server: check if the other servers and drivers are functioning correctly. If a faulty one is detected, it is automatically replaced.

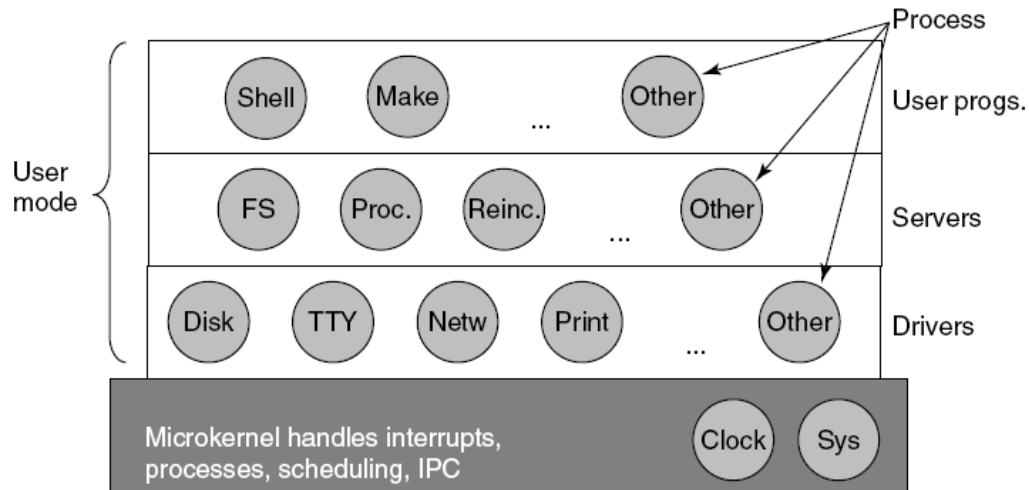


Figure 1-26. Structure of the MINIX 3 system.

- Put the mechanism for doing something in the kernel but not the policy.

Operating System Structure (11)

■ Microkernels

□ Advantages

- Expansion of the system is easier. It is simply added in the system application without disturbing the kernel.
- Reliability
 - Less code is running in kernel mode.
- Portability
 - Changes needed to port the system to a new processor is done in the microkernel, not in the other services.

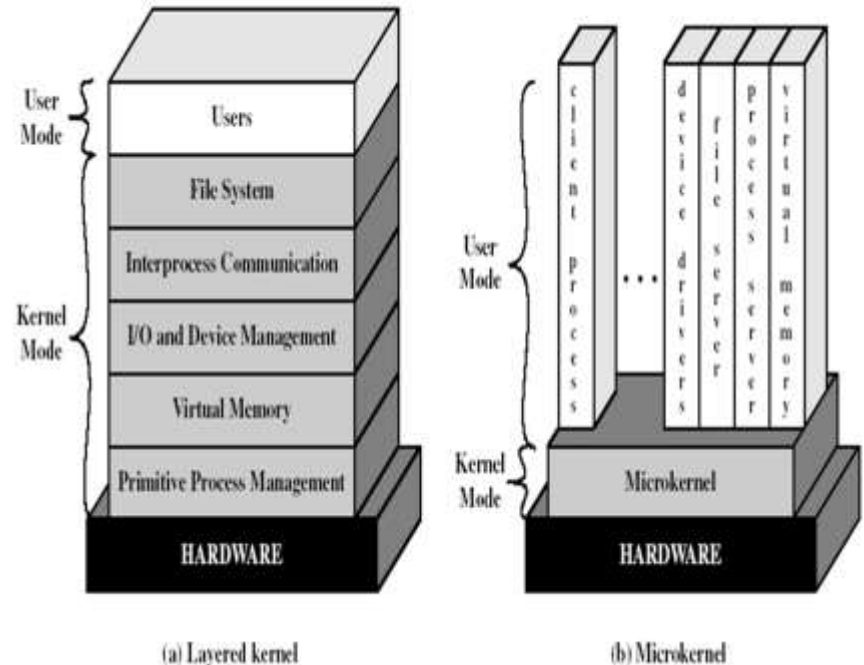
□ Disadvantage

- Slower message passing between user-level processes can affect performance, especially in high-performance applications.
- Increased complexity due to the modular design can make it more difficult to develop and maintain the operating system.

Operating System Structure (12)

■ Layered vs. Microkernels

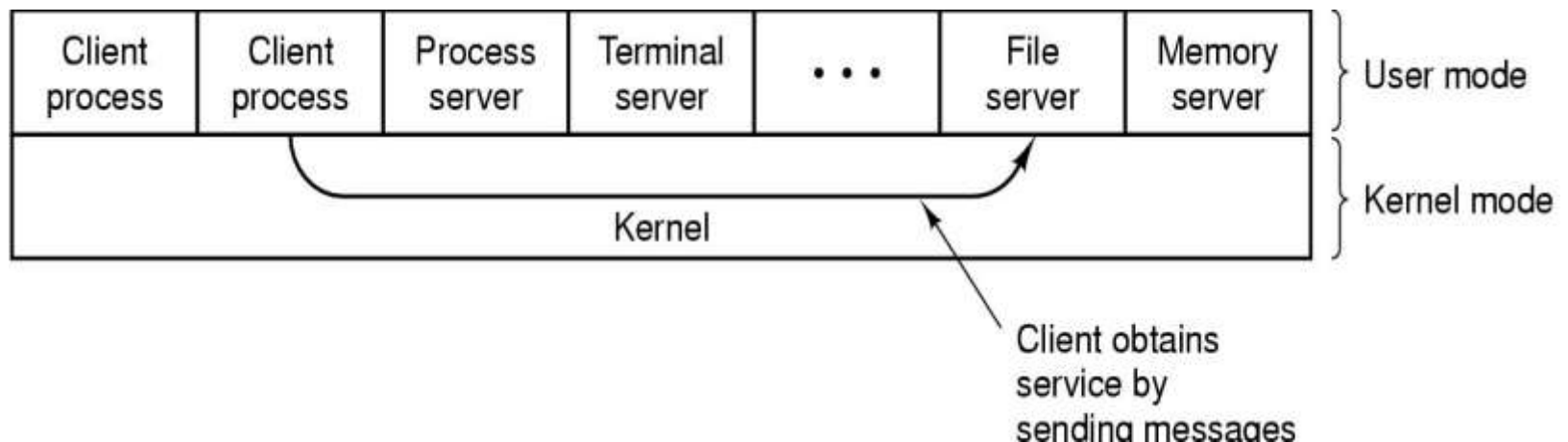
- A Layered Operating System is similar to a Microkernel Operating System in that they both separate functionality into different layers.
- However, a Microkernel Operating System takes the concept a step further by moving more functionality out of the kernel and into user space, which can make it easier to update and maintain the system. In contrast, a Layered Operating System typically keeps more functionality in the kernel.



Operating System Structure (13)

■ Client-Server Model

- A slight variation of microkernel.
- Distinguish 2 classes of processes: the servers and the clients.
- Moving most of the code up into the higher layers made the kernel to be minimal and only responsible for communication between clients and servers.



Operating System Structure (14)

■ Client-Server Model (ctd.)

- The operating system is collection of servers that provide specific services; e.g., file server, etc.

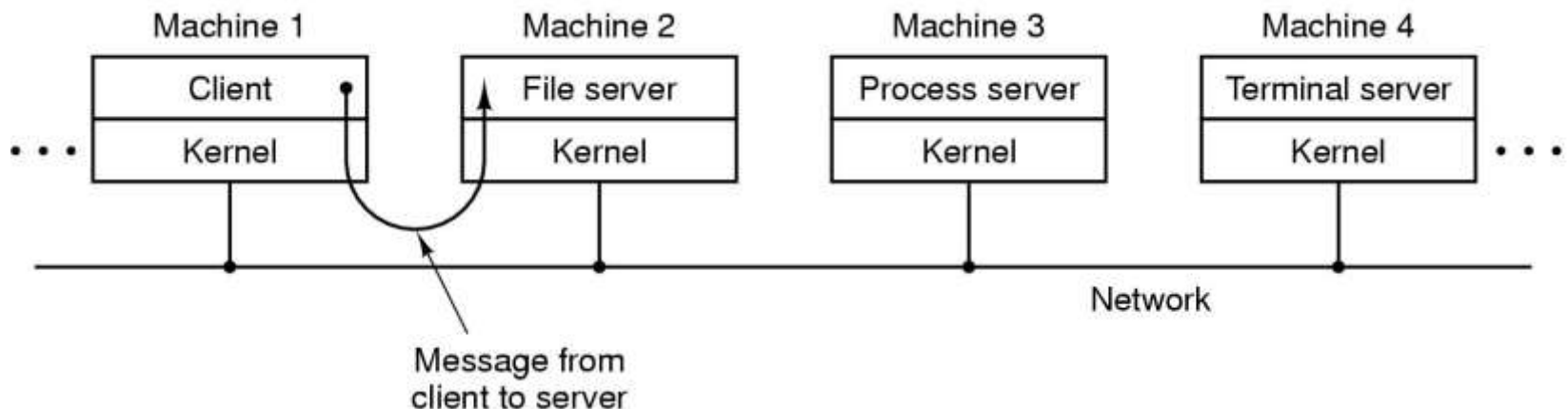


Figure 1-27. The client-server model in a distributed system

Operating System Structure (15)

■ Client-Server Model (ctd.)

□ Advantage

- Adaptability to use in distributed systems.

□ Disadvantage

- Doing OS functions (e.g. loading physical I/O device registers) in the user space is difficult.
- There are two ways to solve this problem
 - To have some critical server processes run in the kernel with complete access to hardware but still communicating with other normal processes.
 - Enhance the kernel to provides these tasks but server decides how to use it (Splitting policy and mechanism).

Operating System Structure (16)

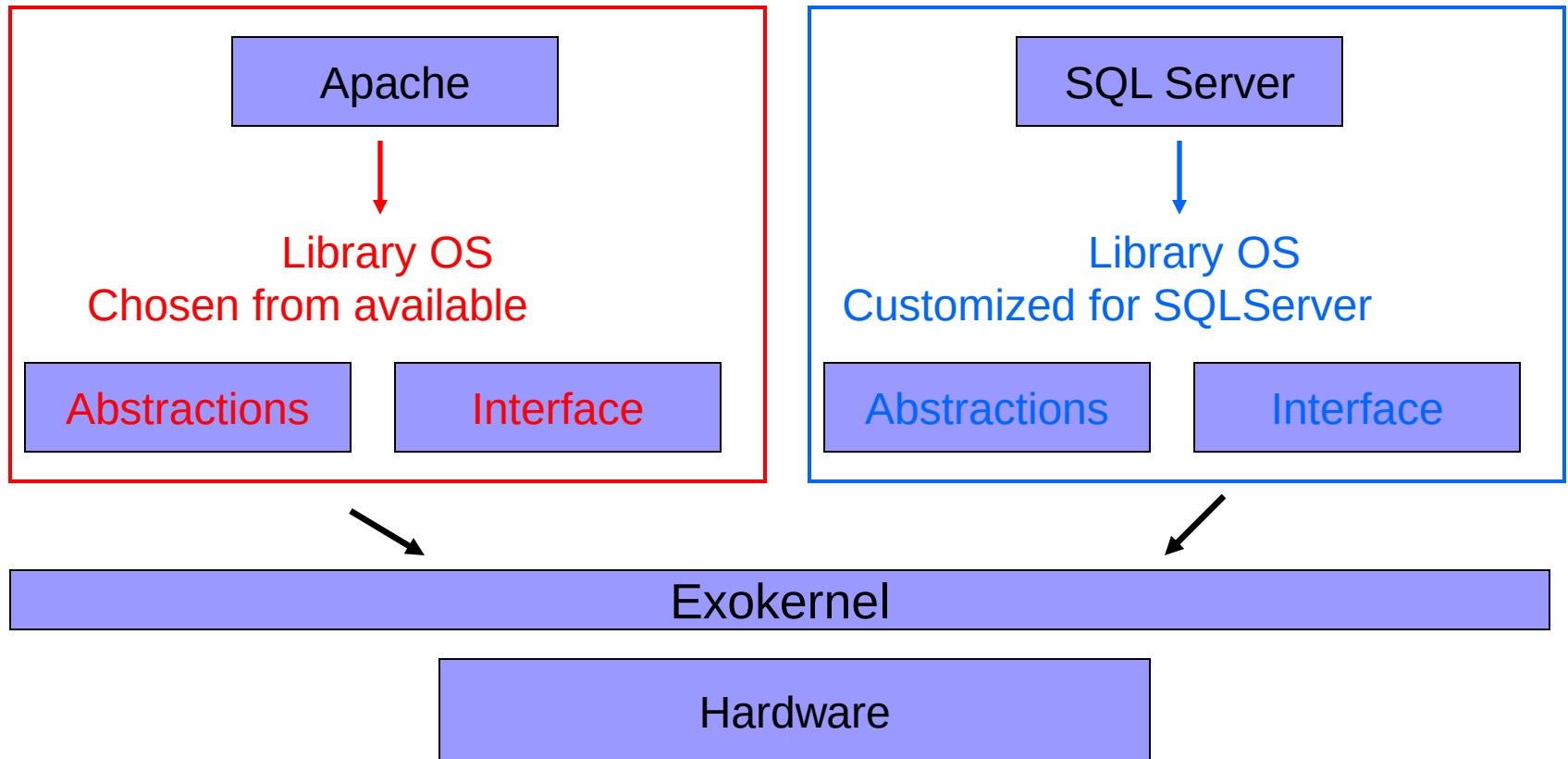
■ Hybrid-kernel

- A combination of both monolithic-kernel structure and micro-kernel structure.
- Example:
 - Windows NT (Windows XP, Windows 7, Windows 8..), Mac OS
- Disadvantages
 - It increases overall complexity of system by implementing both structure (monolithic and micro) and making the system difficult to understand.
 - The layer of communication between micro-kernel and other component increases time complexity and decreases performance compared to monolithic kernel.

Operating System Structure (17)

■ Exokernel

- Developed at MIT to provide application-level management of hardware resources.





Summary (1)

- What is OS
 - OS is an extended machine
 - OS is a resource manager
- OS History
 - Batch systems
 - Multiprogramming
 - Spooling
 - Time sharing
- Operating System Concepts
 - shell



Summary (2)

- System Calls
- Operating System Structure
 - Monolithic System
 - Layered System
 - Microkernel
 - Client-Server Model
 - Exokernel



Homework

- P81 1,3
- Reading Assignment: P713-P732 10.1, 10.2